

NAME:- BHAVANI SANKAR

REG.NO:- 192411238

SESSION 3 LAB EXPERIMENTS

4. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move

the ball out of the grid boundary in exactly N steps.

Example:

· Input: $m=2, n=2, N=2, i=0, j=0$ · Output: 6

· Input: $m=1, n=3, N=3, i=0, j=1$ · Output: 12

CODE:-

```
#include <stdio.h>
```

```
int m, n, N;
```

```
int solve(int i, int j, int steps) {  
    if (i < 0 || i >= m || j < 0 || j >= n)  
        return 1;
```

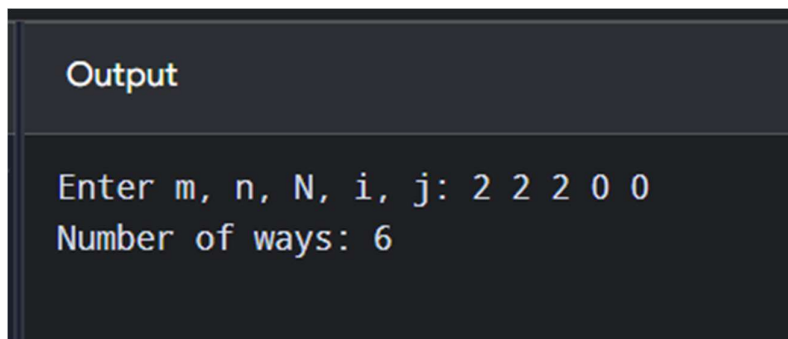
```
    if (steps == 0)  
        return 0;
```

```
    return solve(i - 1, j, steps - 1) +  
           solve(i + 1, j, steps - 1) +  
           solve(i, j - 1, steps - 1) +  
           solve(i, j + 1, steps - 1);
```

```
}
```

```
int main() {  
    int i, j;  
  
    printf("Enter m, n, N, i, j: ");  
    scanf("%d%d%d%d%d", &m, &n, &N, &i, &j);  
  
    printf("Number of ways: %d\n", solve(i, j, N));  
  
    return 0;  
}
```

OUTPUT:-



```
Output  
Enter m, n, N, i, j: 2 2 2 0 0  
Number of ways: 6
```

5. You are a professional robber planning to rob houses along a street. Each house

has a certain amount of money stashed. All houses at this place are arranged in a

circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

CODE:-

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int robLinear(int a[], int start, int end) {  
    int prev1 = 0, prev2 = 0, i, current;
```

```
    for (i = start; i <= end; i++) {  
        current = max(prev1, prev2 + a[i]);  
        prev2 = prev1;  
        prev1 = current;  
    }
```

```
    return prev1;  
}
```

```
int main() {
```

```

int a[100], n, i, result;

printf("Enter number of houses: ");
scanf("%d", &n);

printf("Enter money in each house:\n");
for (i = 0; i < n; i++)
    scanf("%d", &a[i]);

if (n == 1)
    result = a[0];
else
    result = max(robLinear(a, 0, n - 2),
                robLinear(a, 1, n - 1));

printf("Maximum money: %d\n", result);

return 0;
}

```

OUTPUT:-

```

Output

Enter number of houses: 4
1 2 3 1
Enter money in each house:
Maximum money: 4

```

6. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

Input: n=4 Output: 5

Input: n=3 Output: 3

CODE:-

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    int a = 1, b = 2, c;
```

```
    printf("Enter number of stairs: ");
```

```
    scanf("%d", &n);
```

```
    if (n <= 0) {
```

```
        printf("Number of ways: 0\n");
```

```
    } else if (n == 1) {
```

```
        printf("Number of ways: 1\n");
```

```
    } else if (n == 2) {
```

```
        printf("Number of ways: 2\n");
```

```
    } else {
```

```
        for (i = 3; i <= n; i++) {
```

```
            c = a + b;
```

```
            a = b;
```

```
            b = c;
```

```

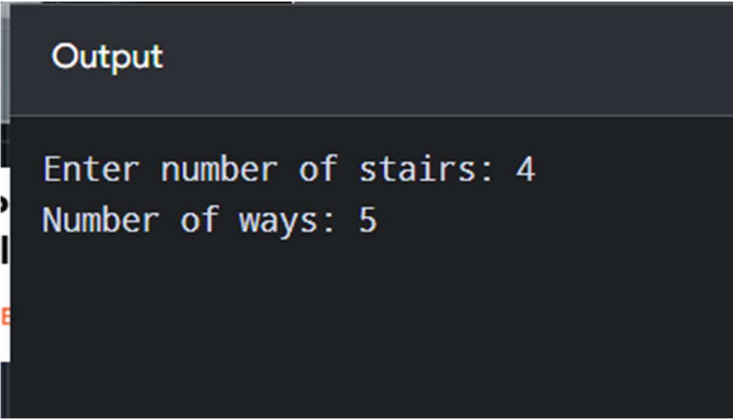
    }

    printf("Number of ways: %d\n", b);
}

return 0;
}

```

OUTPUT:-



```

Output
Enter number of stairs: 4
Number of ways: 5

```

7. A robot is located at the top-left corner of a $m \times n$ grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

Input: $m=7,n=3$ Output: 28

Input: $m=3,n=2$ Output: 3

CODE:-

```
#include <stdio.h>
```

```
int main() {  
    int m, n, dp[100][100];  
    int i, j;  
  
    printf("Enter rows and columns: ");  
    scanf("%d%d", &m, &n);  
  
    for (i = 0; i < m; i++) {  
        for (j = 0; j < n; j++) {  
            if (i == 0 || j == 0)  
                dp[i][j] = 1;  
            else  
                dp[i][j] = dp[i - 1][j] +  
                    dp[i][j - 1];  
        }  
    }  
  
    printf("Number of unique paths: %d\n",  
        dp[m - 1][n - 1]);  
  
    return 0;  
}
```

OUTPUT:-

Output

```
Enter rows and columns: 7 3  
Number of unique paths: 28
```